

open-cake-ir

Agent 驱动的 GPU Kernel 与编译器协同演进

技术报告

A Technical Report on Agent-Driven GPU Kernel
and Compiler Co-Evolution

秦海岩 Haiyan Qin

北京航空航天大学

版本与证据边界。本报告整理仓库在 2026 年 9 月 23 日的实现与已发布证据；正文所读源码固定为 0fe3a4476765a1d323e9d114d1c9af2f324dd767。历史设备实验仍绑定各自的源码、精确目标、Workload、oracle 和计时协议，不能因本报告的新版本而重新归属。文内给出固定提交的来源链接。

2026 年 9 月 23 日

摘要

GPU Kernel Agent 可以快速提出候选实现，但候选为何非法、何处消耗资源、哪次设备测量可以支持收益，往往分散在代码生成器、运行时和实验记录中。open-cake-ir 将单 Kernel 的硬件执行决策表示为带类型的 *Schedule*，以 *Program* 组合阶段；Compiler 在生成目标源码前给出局部化诊断，Lab 在冻结的 Workload 和 Run 合同下搜索候选，Evaluation 以外部 oracle、目标计时和 profiler 检查结果，Evidence 保存可审计的观察。编译器缺口由运行之外的后继提交修复，并通过语料门验证，形成 Kernel 优化与编译能力演进的两个循环。

本文给出系统的职责、表示、检查和实验协议，并将证据按层级陈述。当前源码的 179 项 Corpus Gate 用例符合预期；B300、C550 等目标已有范围明确的设备案例，包括合格的局部改进与未通过计时质量门的完整 Program。现有记录支持平台接入和若干固定任务的局部优化；跨硬件经验材料或显式变换能否降低 Agent 搜索成本，仍需按预注册的四组处理实验验证。本文不将软件合同、设备正确性和因果收益混为一项结论。

关键词：GPU Kernel；中间表示；静态验证；Agent；实验可复查性；跨硬件优化

English abstract. open-cake-ir is a research system in which agents author typed, hardware-explicit kernel schedules, receive localized compiler feedback, and validate candidates against external oracles and target-specific measurements. The compiler evolves between frozen optimization runs, with corpus-gated changes. This report separates software contract coverage, device correctness, qualified local performance results, and the still untested causal claim of cross-hardware knowledge transfer. Its implementation snapshot is commit 0fe3a447; each historical measurement retains its own source and assay boundary.

目录

1	问题、研究对象与论点	3
2	系统合同与职责边界	3
2.1	从任务到可审计结论	3
2.2	两个时钟：候选搜索与编译器演进	4
3	表示：Workload、Program 与 Schedule	4
3.1	显式的计划，而非新的布局代数	4
3.2	Target 是精确的硬件文档	4
3.3	表示变更的验收义务	5
4	Compiler：局部化检查到目标源码	5
4.1	分层拒绝与反馈	5
4.2	源码生成与下游工具链	5
4.3	Corpus Gate 的含义	5
5	实验方法与证据等级	5
5.1	一次 Run 的四阶段	5
5.2	研究对照的单位和参考访问	6
5.3	结论阶梯	6
6	实现覆盖与代表性设备案例	7
6.1	平台覆盖是分层事实	7
6.2	案例 A：C550 的局部 tiling 收益	7
6.3	案例 B：B300 的对齐条件与质量门	7
6.4	案例的合取边界	7
7	跨硬件知识迁移：机制与待验证估计量	8
7.1	可复用的单位与负责位置	8
7.2	E/P 四组实验	8
8	限制、可复查性与结论	8
	阅读与引用说明	9

1 问题、研究对象与论点

给定同一数学算子，GPU 实现仍须决定任务分块、线程分工、存储层次、访问坐标、同步与执行顺序。一个只输出延迟的搜索过程很难说明某次失败属于候选调度、编译器表达能力、硬件准入，还是测量噪声。CAKE 论文提出让 Agent 编辑显式执行计划，并以编译器分析和外部评测形成反馈闭环 [1]。open-cake-ir 是对这一研究路线的独立工程实现；论文报告的模型实验和速度数字不作为本仓库结果 [6]。

本报告的研究对象是可复查的 *Agent* 编写环境：一份候选包含什么决策，系统能在什么范围内拒绝，设备证据如何决定接受，以及真实缺口怎样改变下一版 Compiler。核心问题可以表述为：

在保持 Workload 语义和实验条件可追溯的前提下，怎样让 Agent 的 Kernel 决策可检查、让失败有负责位置，并让重复经验进入后继编译能力？

本文主要陈述三项已实现的机制：显式的 **Program/Schedule** 合同、分层诊断与目标精确准入、冻结运行和追加式证据。跨硬件知识迁移的增量价值是后续实验问题，不是现有软件实现的推论。

2 系统合同与职责边界

2.1 从任务到可审计结论

Workload Contract 固定数学语义、输入域、外部 oracle 和容差。Schedule 描述一个 Kernel 的资源、执行组、操作、地址与同步；Program 再声明公共张量、各阶段顺序和绑定。RunSpecification 固定一次优化所用的源码、目标、可见材料、工具权限和预算。StudyPlan 仅在研究对照时进一步固定处理、分配、终点与分析。这些对象的职责不同，不能用一个候选的局部通过来代替 Study 结论 [2, 11]。

表 1: 一次候选从表达走向结论时的负责位置。箭头表示依赖方向，不表示某层的通过自动推出下一层的通过。

对象或模块	拥有的事实	对外给出的产物
Workload	数学、形状、输入分布、外部答案	不随候选改变的验收合同
Compiler	IR 类型、Target、合法性、后端准入与生成	Assessment、Finding、目标源码
Lab	材料、候选搜索、预算与 Run 控制	已提交候选和过程事件
Evaluation	外部正确性、目标计时质量和剖析	独立的检查、计时和 profiler 收据
Evidence	原始文件、事件、回放与审计	可由原始材料重建的报告
Study	处理分配、估计量和缺失策略	限定研究问题的分析结果

Compiler 是独立产品核心，不导入 Lab、provider、Workload 实现或证据库。Lab 调用

Compiler 与 Evaluation, Evidence 保存其观察。这样的依赖关系使同一个 Compiler 可用于离线检查, 也防止实验策略偷偷进入 Schedule 的语义判断。系统源码与边界见[架构章节](#)和[职责地图](#)。

2.2 两个时钟：候选搜索与编译器演进

内循环在冻结的 Compiler、Workload、Target、oracle 和测量合同下进行：生成结构不同的候选, 先通过构造检查、Verifier 和后端预检筛选, 再对幸存者做设备正确性、计时与剖析, 最后把诊断送回下一次候选。外循环由累积的 Finding 触发：若问题属于通用合法性、缺失原语、lowering 或目标校准, 维护者在后继提交中修改其负责模块, 验证后才开启新 Campaign。一次运行不能边测边改变尺子 [2, 5]。

3 表示：Workload、Program 与 Schedule

3.1 显式的计划，而非新的布局代数

受限 Python 前端与 JSON 文档形式进入同一规范的 typed Schedule；Python 入口解析受支持的语法, 并不执行任意宿主 Python 程序。Schedule 记录执行组、缓冲区、访问映射、循环、操作与同步, 性能相关的硬件选择因此可被检查。它没有把 layout 建成一阶代数；对目标硬件的具体存储与访问承诺由相应 Target 和后端解释并验证。Triton 路线仍把部分物理 placement、寄存器分配与指令选择交给下游工具链, 不能把其生成结果当作 Schedule 已经精确指定的事实 [3, 2]。

例如对一行 Softmax, Workload 给出 $y_j = \exp(x_j - m) / \sum_k \exp(x_k - m)$, 其中 $m = \max_k x_k$ ；候选 Schedule 另选每个执行组处理的行、列块、归约方式与写回坐标。若读取、归约和写回对同一行的坐标约定不一致, 问题属于数据一致性；若两个执行组写同一输出, 问题属于程序安全；若目标不提供所需指令, 则属于硬件符合性或后端资格。数学式相同不能消除这些执行计划差异。

Program 负责多个 Schedule 的公共输入输出、阶段顺序与张量绑定。完整 Program 的正确性要检查所有阶段的最终输出；单 Kernel 的计时或剖析不能自然推广为 Program、模型或服务收益。

3.2 Target 是精确的硬件文档

当前源码声明八个精确目标, 分属 NVIDIA、Apple、AMD、Hygon 和 MetaX 五个 Vendor。Target 文档拥有自己的 lane 宽度、存储空间、指令合同、代码对象及资源事实；未声明事实给出“未建模”的报告, 不借用相似目标的常数。Backend 是生成机制而不是 Vendor：现有 triton、metal、native_cuda 和 cutlass_cute_dsl 各声明可生成的代码对象。Compiler 对目标和后端做精确匹配, 不能把未声明目标降级到邻近架构 [4, 2]。

3.3 表示变更的验收义务

新增 IR 原语须同时给出类型、效应、合法性、静态分析和目标行为；只有语法而没有这些语义，不能成为已准入能力。仓库将易编写、性能决策可见、规范形式、构造时类型检查、可分析、语料门、分析一致与硬件根据作为表示的八项设计约束 [1, 3]。某算子若能由现有原语组合实现，应先添加 Workload Contract；只有真实表达缺口才要求扩大 Compiler。

4 Compiler：局部化检查到目标源码

4.1 分层拒绝与反馈

Compiler 首先解析与构造类型正确的 IR，然后以四类合同检查：schedule_semantics、hardware_conformance、data_consistency 和 program_safety。Assessment 中的 Finding 指向字段或源码区域，带稳定代码、类别、严重度和阻断范围；非阻断的 guidance 与拒绝原因分开。后端预检负责该生成路线特有的能力边界。静态分析只能在已建模的范围内阻断；未声明的资源上限和下游产生的物理资源需求需要另行观察 [2, 5]。

一个可复查的小反例是 FMA 候选漏掉第三个输入：构造仍指向同一数学操作，Assessment 在 operations[3].reads 报 ELEMENTWISE_ARITY，指出该操作需要三个输入。作者因此可以修复具体数据边，无需先耗费设备时间。通过静态检查只允许候选继续；它不证明工具链编译、GPU 数值正确或速度收益 [2]。

4.2 源码生成与下游工具链

合法 Schedule 经选定后端输出可检查的目标源码。Triton 路线再通过厂商对应的 Triton 工具链编译；Metal、原生 CUDA 与 CuTe DSL 路线各保留自己的预检与生成条件。Target 声明的代码对象随生成路线进入工具链和 Executor，不能由目标名字猜测架构。生成源码、编译产物、设备可加载性、外部正确性与实测延迟是五个递进但互不等同的检查点 [2]。

4.3 Corpus Gate 的含义

固定源码 0fe3a447 的状态页报告 179/179 项语料检查符合已审查预期；其中 Apple family7 与 family9 是已声明但未被该语料检查的目标。该数值证明当前软件合同与预期相符，不证明这些目标的设备运行或性能。语料预期必须作为独立审查决定维护，不能为使 Gate 变绿而在验证前重生成 [7, 5]。

5 实验方法与证据等级

5.1 一次 Run 的四阶段

系统按下列顺序运行候选搜索：

1. 生成结构不同的 Schedule 或 Program 候选，并记录具体变更；

2. 以 IR 构造、Verifier、backend preflight 和有目标覆盖的成本估计筛选，未覆盖时明确弃权；
3. 对幸存者使用外部 oracle 检查完整输出，再在目标计时器、状态重置与质量门下测量，并保留 profiler 观察；
4. 将失败归于候选、Verifier、成本模型、IR 或 lowering，决定继续搜索或登记后继 Finding。

成本估计用于设备时间之前的排序与过滤，不能决定最终接受。每个目标分别声明计时器、测量区间和样本前设备状态；缺少其中一项时应报告测量覆盖限制，而不是继承其他目标的延迟语义。

5.2 研究对照的单位和参考访问

正式比较的实验单位是独立 Run，不是 Turn、候选或检查点。干净起点只能看到规格、oracle、高层代码和被授权的参考资料，不能读取目标的完整低层实现；已知 Kernel 复现则可以读取该实现。直接低层编写与 Cake 编写是 Authoring Environment 的不同处理条件，需要共同的目标、任务、模型、预算与评测协议。搜索内的最佳读数须经候选固定后的独立确认；基础设施故障应保留为缺失，不能重写成数值错误 [6, 11]。

5.3 结论阶梯

表2 是本文解读每个案例的最小证据门。任一层的通过只授予该层的陈述资格。

表 2: 结论阶梯；“通过”不沿表格自动向下传递。

可陈述的结果	必要证据	尚需另行验收
结构与目标准入	typed IR、精确 Target、Finding 与后端预检	工具链编译和加载
设备正确性	完整输出对外部 oracle，覆盖规定形状与分布	计时质量和性能收益
局部性能	固定候选与基线、共同计时区间、质量通过	跨任务或跨硬件泛化
Program 或服务收益	完整调用路径、dispatch 与应用层验收	Agent 处理的因果效果
研究归因	预注册处理、独立 Run、共同确认与缺失规则	未测试任务与平台的外推

Evidence 保存候选、原始样本和事件；报告是由这些记录派生的视图。源码身份由干净 Git 提交决定，Compiler 为 open-cake-ir@<commit>，Executor 为 <target>@<commit>。本报告的阅读提交只界定当前叙述所见的软件，不能代替历史实验的执行提交或文件监管条件。

6 实现覆盖与代表性设备案例

6.1 平台覆盖是分层事实

表3 概括仓库已发布的代表性证据。它不是跨厂商性能排行；同一平台的不同 Workload 也可能有不同的计时和确认资格。完整细节由各平台专题与原始收据负责 [2]。

表 3: 截至报告快照的代表性平台能力与主要解释边界。

平台	已报告的执行范围	需保留的边界
NVIDIA	B300 的单 Kernel、部分完整 Program； B200 另有固定案例	FlashInfer 外部比较有通过与未通过质量门的边，按任务解读
Apple	Metal 固定算子与 TaskLab 优化记录	已验收设备和计时口径不推广到全部 family 或完整 Program
AMD	gfx1151 设备调查与 smoke	计时器绝对时间未对齐，尚无合格的完整 Agent 收益结论
Hygon	BW1101 / gfx938 固定任务与 HIP 路线	短 Kernel 的计时分辨能力限制部分比较
MetaX	C550 固定单 Kernel 及若干完整输出；单 Kernel 配对计时和 profiler	完整 Program 普通 Run 计时与完整 provider 闭环仍需验收

6.2 案例 A: C550 的局部 tiling 收益

在 C550 的 FP16 GEMM 固定形状 $M = 17, N = 128, K = 2048$ 上，已验收基线保持不变，显式地把 M tile 从 64 改为 32。独立确认的基线和候选中位数分别为 **72.448** 与 **58.368 μs** ，比值 **1.241** 倍；正确性 case 与测量质量门通过。这是 local_serialized 范围内的本机 authoring 对照，支持这个形状上的局部调度改进。它没有比较“向 Agent 提供 NVIDIA 经验”与“不提供经验”的处理差异，也不是完整 Program 的自动优化 Run[9]。同一专题中的 17 个 Workload 变体、86/86 个完整输出 case 属于正确性证据；其计时为缺失，不能拿来扩大上述收益范围。

6.3 案例 B: B300 的对齐条件与质量门

B300 FlashInfer 任务 026 的守卫式对齐候选，相对同次冻结的 generic sliced-w8 对照得到合格的 **1.081** 倍局部收益；比较所用中位数为 6.4005 与 5.920 μs 。同一候选对外部参考的边没有通过 CV 质量门，因此不能从该案例推出对外部参考的合格胜负。另一任务 008 的完整两阶段 Program 已通过 2550 份数值快照，但新候选相对对照及外部参考的计时边均未通过质量门：它证明完整程序可运行和算对，不提供合格的性能结论 [8]。

6.4 案例的合取边界

两个局部改进来自不同硬件、Workload、源码与计时机制，不能平均，也不能作为一个跨硬件机制已迁移的轨迹。负结果、close_null、未完成和质量失败与正结果同样保留。截至此报

告所读的[当前状态页](#)，项目没有统一的已验收 Study Report 索引；本文不计算全项目成功率或总加速比。

7 跨硬件知识迁移：机制与待验证估计量

7.1 可复用的单位与负责位置

候选优化可能包含两种不同的知识：一个带适用条件的语义改写，以及在某个硬件上选出的 tile、执行组或 pipeline 参数。前者可在 IR 中形成显式 pass，声明前提并返回完整候选或拒绝原因；后者要在目标 Target 上重新选择和验证。以私有中间张量的融合为例，数据流私有性、舍入、消费者和生命周期是机制条件；寄存器占用与具体收益属于目标设备事实。来源设备的速度比不随机制迁移 [10]。当一次研究没有可推广改写时，记录 No promotion 是正常结果。

7.2 E/P 四组实验

仓库已实现对经验材料 E 与显式变换调用权 P 的四格隔离和软件协议，但尚没有共同目标、基线与预算下的实机迁移收益实验。四组分别为 $E0P0$ 、 $E1P0$ 、 $E0P1$ 和 $E1P1$ ； $P1$ 的最小接口合同本身会暴露必要机制信息，因而 E 的效果只指额外说明与案例。各组共享 typed IR、Target、oracle、计时质量门和可表达的基础能力。材料与调用权限须通过真实隔离执行，不能只由提示词声称关闭 [11]。

令 p_{ep} 为在预注册预算内经新鲜确认、达到本机预定改善阈值的 Run 比例。预定主要对比为

$$\begin{aligned}\Delta_E &= \frac{1}{2}[(p_{10} - p_{00}) + (p_{11} - p_{01})], \\ \Delta_P &= \frac{1}{2}[(p_{01} - p_{00}) + (p_{11} - p_{10})], \\ \Delta_{EP} &= p_{11} - p_{10} - p_{01} + p_{00}.\end{aligned}$$

独立 Run 是重复单位；正式多任务研究以任务等权汇总并保留组内配对结构。搜索成本、失败、预算耗尽和基础设施缺失须按预注册规则保留。协议的存在证明问题可被提出和执行，只有冻结处理后的设备实验才能估计上述三个量。

8 限制、可复查性与结论

本文的最强软件结论是当前源码上的表示、检查、生成和控制协议，以及 179 项语料预期相符。最强设备结论是指定源码、目标与固定 Workload 上的正确性和局部合格改善。当前材料不足以估计跨硬件经验迁移的因果收益、普遍性能优势，或模型与服务层面的端到端收益。静态分析的覆盖也受目标声明和后端实际资源分配限制；缺乏目标计时校准时应报告未知，而不是给出没有根据的延迟预测。

后续工作以同一职责边界推进：补齐各平台完整 Program 与目标框架验收；在有原始证据的目标上校准测量与成本估计；将重复的失败形成有反例的 Verifier 规则、IR 能力或 guarded

pass；最后用预注册 E/P 实验检验迁移增量。这个顺序让 Kernel 结果、Compiler 演进与研究归因各有自己的证据和版本。

阅读与引用说明

本文是作者维护的仓库技术报告，无 DOI、期刊或同行评审声明。引用设计时请固定本报告及其所读源码提交；引用性能时还需追溯案例自己的执行源码、精确目标、Workload、baseline、oracle、计时区间与质量门。以下文献中的仓库链接均固定到本文所读的 0fe3a447，而实验原始收据的身份由各专题继续给出。

参考文献

- [1] Z. Ye et al. *CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution*. arXiv:2608.12629v1, 2026. <https://arxiv.org/html/2608.12629v1>.
- [2] H. Qin. 系统架构与平台能力, open-cake-ir, fixed source commit.
- [3] H. Qin. IR 指南, open-cake-ir, fixed source commit.
- [4] H. Qin. Compiler 清单与 Target 文档示例, open-cake-ir, fixed source commit.
- [5] H. Qin. 验收门, open-cake-ir, fixed source commit.
- [6] H. Qin. 论文与本地证据合同, open-cake-ir, fixed source commit.
- [7] H. Qin. 当前状态, open-cake-ir, fixed source commit.
- [8] H. Qin. B300 FlashInfer 任务综述, open-cake-ir, fixed source commit.
- [9] H. Qin. C550 专题与 M17 独立确认, open-cake-ir, fixed source commit.
- [10] H. Qin. 跨硬件优化知识迁移机制, open-cake-ir, fixed source commit.
- [11] H. Qin. E/P 消融实验设计, open-cake-ir, fixed source commit.